Handwritten digit recognition with 3,000 images

Does a classifier learn more from a synthetic digit than from seeing the same original a second time?

We extended the Assignment 1 (A1) digit classifier with 1,000 synthetic derivatives of public MNIST training images. The first pilot supplied 120 examples. We added another 880, or 88 per digit. The resulting dataset contains 300 images for each of the ten classes.



d0-v013 d2-v013 d3-v013 d4-v013 d7-v013 d9-v013

Figure 1. Examples from the expansion. Each has its own MNIST training reference. The sample ID links the image to its source, generator settings and label provenance.

Comparison at a glance

Training condition	Correct on same test	Test macro-F1
Original A1	295 / 300	98.35%
A2 with 1,000 synthetic digits	**296 / 300**	**98.67%**
Matched real-only fine-tuning	296 / 300	98.68%

Both fine-tuned classifiers started from the saved A1 weights. They saw the same 1,400 originals in the same order. Where the augmented model saw a synthetic image, its control saw the exact real handwriting used to generate it. We used the validation split to select the demo model.

How the dataset reached 3,000

A1 comprises a balanced subset of public MNIST, with source IDs, labels and pixel hashes [1]. We preserved the project's validation and test splits. Every synthetic image comes from a different training original. Illustrations of a digit and its derivative therefore stay on the same side of the train/test split.

Split	MNIST	Pilot	Expansion	Total
Training	1,400	120	880	2,400
Validation	300	0	0	300
Test	300	0	0	300
All images	**2,000**	**120**	**880**	**3,000**

Per class, training has 140 original digits and 100 synthetic digits. Validation and test each have 30 real digits per class. Synthetic examples constitute **33.3%** of the entire dataset and **41.7%** of the training split.

Three ways to vary a handwritten stroke

Thin stroke (336)



Source Derivative

Thick stroke (332)



Source Derivative

Rotation (332)



Source Derivative

Figure 2. Counts refer to all 1,000 synthetic images. Stroke thickness and rotation are set before neural editing. Each pair above uses a different source.

Who supplied the labels?

The reviewer approved the 120-image pilot in a batch and retained its source labels. For the new 880, the pipeline inherits the label from the verified original. A visual screen flagged severely ambiguous shapes for replacement. The index records which images were batch-approved and which inherited their source label.

How TA-TiTok and MaskGen-L edit a digit

TA-TiTok compresses an image into an ordered sequence of visual tokens. Its decoder uses the tokens and text to reconstruct an image. MaskGen learns to refill missing tokens given the visible tokens and a description. We used the authors' released VQ (discrete-token) checkpoints for TA-TiTok BL128 and MaskGen-L [2-4].

From one training digit to one derivative

1. **Prepare the source**
   Centre a handwritten MNIST training digit on a 256 x 256 white canvas.
2. **Change the writing style**
   Thin or thicken the stroke, or apply a small rotation of 8 or 14 degrees.
3. **Refill masked tokens**
   TA-TiTok encodes 128 tokens. MaskGen-L uses the digit prompt to replace 4-8.
4. **Apply the learned change**
   Decode both sequences under the same prompt. Add their difference near the stroke.

Figure 3. The published models supply the encoder, token predictions and decoder. The source choice, style controls, partial resampling and stroke-local rendering are project adaptations.

Why start from a known shape?

In the original text-only pilot, a prompt for a digit sometimes produced a pen or a picture of a sheet of paper. The frozen A1 classifier agreed with the requested class in only 14 of 120 cases. After introducing MNIST references, it agreed on 113 of the first 120 derivatives. These counts describe the candidate pool. We report held-out test results on page 5.

For each masked position, we sample a different token among eight high-scoring alternatives. To preserve the background and digit identity, we bound the decoded difference to 64 gray levels and apply it through a mask around the stroke. The deterministic style controls supply most of the apparent change. Model weights and sample seeds are recorded in the released recipes.

A controlled fine-tuning experiment

We reused the A1 CNN, which has three convolutional blocks (32, 64 and 128 filters), max pooling and a fully connected ten-class output. We use the original preprocessing for all scored images. It inverts dark ink, fits the image within 24 x 24 pixels and centres it on a 32 x 32 black field.

Run	First 28 batch slots	Remaining 20 slots
Real-only control	A1 training originals	Matched source originals
Expanded A2	**Same originals, same order**	**Synthetic versions of those sources**

Each epoch has 50 updates. The augmented branch sees each of the 1,400 originals and each of the 1,000 synthetic images exactly once. The control replays the matching original for each synthetic slot. Both branches use the same starting A1 checkpoint, dropout seed and batch order. They run for five epochs with Adam, learning rate 0.0001, weight decay 0.0001 and seed 42.

Learning curves

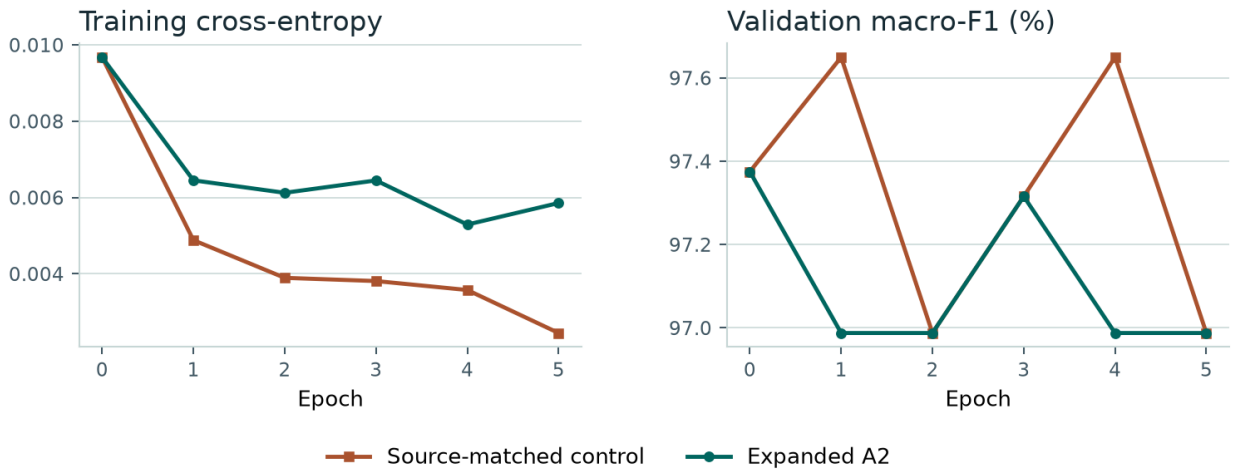


Figure 4. Epoch 0 is the A1 checkpoint. Training loss is evaluated on the original 1,400 images. Each validation point uses the same 300 original images.

For each branch we kept the epoch with the highest validation macro-F1, using lower cross-entropy to break ties. We also retained A1 and the previous 120-image model as deployment candidates. Test predictions were collected only after selecting the epochs and deployment.

Accuracy and macro-F1 on held-out digits

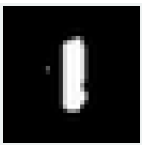
We scored all four models on the same 300 MNIST test images, with 30 examples per class. Accuracy counts the fraction of correct predictions. Macro-F1 computes F1 for each class and averages the ten values, giving each digit equal weight.

Classifier	Val. F1	Test correct	Test accuracy	Test F1
A1 baseline	97.37%	295/300	98.33%	98.35%
A2 pilot (120)	97.65%	296/300	98.67%	98.68%
Real-only control	97.65%	296/300	98.67%	98.68%
A2 expanded (1,000)	**97.32%**	**296/300**	**98.67%**	**98.67%**

Table 1. All F1 values are macro-averages. The real-only control follows the same training schedule as the expanded model. The earlier pilot used a different schedule.

Comparison with Assignment 1

The expanded classifier scored **98.67% accuracy** and **98.67% macro-F1**. Its changes from A1 were **+0.33** and **+0.31** percentage points, respectively. It corrected 3 previous errors and introduced 2 new errors. Each test image accounts for one third of a percentage point of accuracy.



True 1
A1: 8
Expanded: 1



True 5
A1: 5
Expanded: 3



True 3
A1: 8
Expanded: 8

Figure 5. Individual held-out inputs and paired predictions. Left to right, the expanded model fixes an error, introduces an error and repeats an A1 error.

Did synthetic data help beyond extra training?

The two branches made the same number of test errors. The difference was 0.00 points in accuracy and -0.01 points in macro-F1. With one seed and one small test set, this comparison cannot establish an effect that will hold across writers and devices.

Drawing, prediction and explanation

The demo accepts one digit drawn with a mouse or finger. The server crops the visible stroke, adds a margin and applies A1 preprocessing. It returns one predicted digit. The demo uses the **real-only control** model, selected with a validation macro-F1 of **97.65%**.

What influenced the prediction?

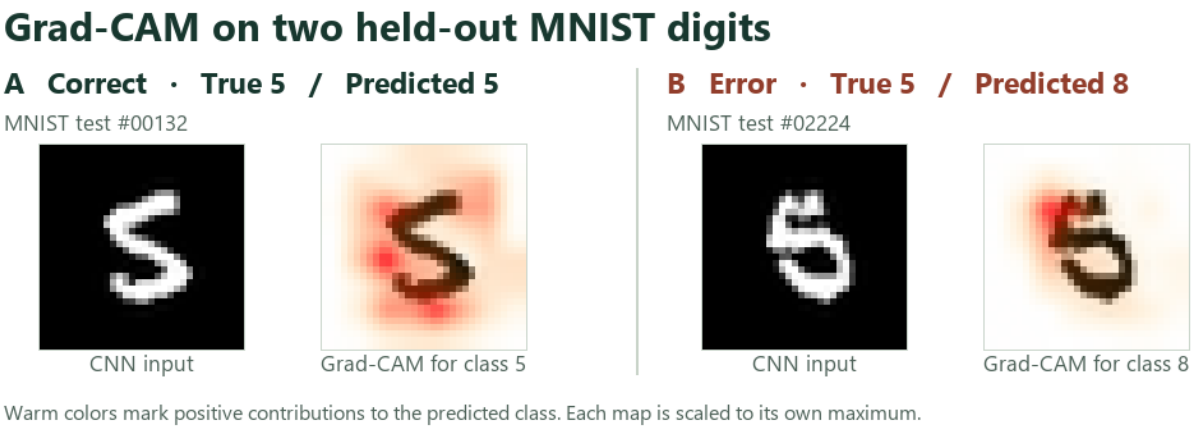


Figure 6. Grad-CAM from the deployed model on real test inputs. Each map represents positive contributions to its own predicted class. Each panel is scaled to its own maximum.

The user can expand the explanation below the answer. Grad-CAM weights the final convolutional feature maps using gradients of the predicted-class score [5]. The model correctly read test image #00132 as 5. It read #02224, a true 5, as 8. The warmest area on that incorrect prediction lies near the short upper-left strokes.

CPU inference time

Measurement	Mean	Median	95th percentile
One digit	0.939 ms	0.925 ms	1.072 ms

After 20 warm-up calls, we timed 100 single-image forward passes on an Intel Core i5-6500 CPU under Windows 10, using PyTorch 2.6.0+cpu and 2 intra-op threads. Input decoding, preprocessing, the Grad-CAM backward pass, network transfer and browser rendering add time outside this measurement.

The browser displays the average forward time, device and browser in the expanded explanation. Drawings are processed locally in memory. The training dataset is distributed separately from the inference interface.

Which examples belong in training?

We generated an initial pool of 880 additional images. Visual inspection flagged 24 severely ambiguous results. A reserve run supplied 24 replacements from previously unused training originals of the same digit and style. The expanded branch uses exactly 880 additional images. The raw attempts and the reasons for exclusions remain auditable.



Original **Excluded variant** **New original** **Selected variant**

Figure 7. Same intended class and style, different training sources. The raw candidate d6-v013 was replaced by d6-v101. The original labels and both generated files remain available in the release records.

Checks for duplicates and leakage

We verified all 3,000 files by encoded PNG, decoded grayscale pixels and the actual 32 x 32 classifier input. The index contains 3,000 distinct network inputs and 0 exact input overlaps across splits. All 1,000 synthetic parents are distinct A1 training images. Repeated handwriting styles or near-duplicates are still possible.

Limits of the evidence

Augmentation reuses existing writers and shapes. It creates more training images without collecting 1,000 independent examples of new handwriting. Source labels can carry inherited ambiguity, and a clean white background simplifies recognition. The experiment measures performance on a balanced MNIST subset. A test with consenting users drawing on touchscreens or with a mouse would measure the intended use more directly.

Attribution, consent and misuse

MNIST is credited to Yann LeCun and Corinna Cortes and derives from NIST. The distributed images retain the CC BY-SA 3.0 attribution and a description of the changes [1]. Each generated entry is marked synthetic. The app assigns a digit even to an unrelated mark, so a prediction or heatmap alone should not authorize a consequential action.

Run the demo and repeat the experiment

Use Python 3.12 and the CPU PyTorch setup in the root README. The repository includes the original PNGs, generated PNGs, classifier checkpoints and metrics. The complete index gives each image's path, split, class, origin and hashes. Run the following commands from the repository root.

Interactive demo
python assignment2/demo.py

Repeat the controlled comparison
python assignment2/train_expanded.py

Export all 3,000 images as a ZIP
python assignment2/export_dataset.py

Open the demo at 127.0.0.1:8766. Under assignment2, outputs/synthetic_1000 contains the index and QA summary. The results_3000 directory contains training history, coverage per epoch, paired predictions and checkpoint hashes. The saved deployment and the expanded-branch checkpoint have separate filenames. The README documents regeneration, including the pinned pretrained weights.

References and release files

[1] Keras. MNIST digits classification dataset.
Dataset source and CC BY-SA 3.0 attribution.

[2] Kim, D., et al. (2025). Democratizing Text-to-Image Masked Generative Models.
ICCV. TA-TiTok and MaskGen with compact text-aware one-dimensional tokens.

[3] Mozgovoy, K. (2025). ByteDance Unveil TA-TiTok Tokenizer. Neurohive.

[4] ByteDance. MaskGen implementation, release notes and checkpoint links.

[5] Selvaraju, R. R., et al. (2017). Grad-CAM. ICCV.

Project run guide and training commands

Full dataset index and split checks

Expanded experiment metrics and paired test predictions